

MSDS 6012 · INGENIERÍA DE DATOS · USFQ

Informe Ejecutivo

Taller en Clase 1 — Parte 5

Flujo ETL Multi-Fuente y Data Warehouse en Modelo Estrella

Estudiante: Tais Rodríguez

Curso: MSDS 6012 — Ingeniería de Datos

Herramientas: KNIME Analytics Platform, MySQL, PostgreSQL, MongoDB, Docker

Repositorio: msds_ingenieria_datos (GitHub)

1 Objetivo

La Parte 5 del Taller en Clase 1 solicita construir un flujo ETL en KNIME que integre **al menos 4 fuentes de datos heterogéneas**, aplique transformaciones reales dentro del entorno de KNIME, y cargue el resultado en un **Data Warehouse con modelo estrella de al menos 5 dimensiones y una tabla de hechos**. A diferencia de la Parte 4 (donde se resolvió el ETL directamente con SQL), aquí el uso de nodos de KNIME es un requisito explícito del ejercicio.

Se decidió mantener el mismo dominio de negocio que la Parte 4 (ventas), extendiéndolo con tres dimensiones nuevas — **cliente**, **canal de venta** y **tiempo** (esta última reconstruida desde un archivo CSV independiente) — para cumplir con el requisito de 4 fuentes tecnológicamente distintas: un archivo plano, una base relacional ya existente, otra base relacional distinta, y una base NoSQL.

2 Fuentes de datos utilizadas

#	Tecnología	Contenido	Filas	Rol en el modelo
1	CSV (archivo plano)	Calendario de fechas (<code>calendario.csv</code>)	32	Dimensión <code>dim_tiempo_p5</code>
2	MySQL (<code>ejerciciodw</code>)	Ventas, productos, regiones y tabla puente cliente/canal	32 / 33 / 33 / 32	Hecho + dimensiones <code>dim_producto_p5</code> , <code>dim_region_p5</code>
3	PostgreSQL (<code>clientes_db</code>)	Catálogo de clientes	10	Dimensión <code>dim_cliente_p5</code>
4	MongoDB (<code>ejerciciodw</code>)	Colección <code>canal_venta</code>	4	Dimensión <code>dim_canal_venta_p5</code>

Como las fuentes de MySQL, PostgreSQL y MongoDB no comparten una llave natural común para relacionar cada venta con su cliente y canal, se construyó una **tabla puente** (`ventas_cliente_canal`) directamente en MySQL, con un mapeo determinista (`id_cliente = ((id_venta-1) % 10) + 1` , `id_canal = ((id_venta-1) % 4) + 1`) que permite unir las 32 ventas con uno de los 10 clientes y uno de los 4 canales de forma reproducible.

3 Diseño del Data Warehouse (Modelo Estrella)

hechos_ventas_360

Columna	Tipo / rol
<code>id_venta</code>	Identificador de la venta
<code>id_tiempo</code>	FK → <code>dim_tiempo_p5</code>
<code>id_producto</code>	FK → <code>dim_producto_p5</code>
<code>id_region</code>	FK → <code>dim_region_p5</code>
<code>id_cliente</code>	FK → <code>dim_cliente_p5</code>
<code>id_canal</code>	FK → <code>dim_canal_venta_p5</code>
<code>cantidad</code> , <code>precio_unitario</code> , <code>monto_total</code>	Medidas

dim_tiempo_p5

32 filas · CSV

dim_producto_p5

33 filas · MySQL

dim_region_p5

33 filas · MySQL

dim_cliente_p5

10 filas · PostgreSQL

dim_canal_venta_p5

4 filas · MongoDB

hechos_ventas_360

32 filas · Hecho (5 FKs + 3 medidas)

El diseño cumple el requisito mínimo de 5 dimensiones + 1 tabla de hechos. Se usó el sufijo `_p5` en las dimensiones que replican conceptos ya existentes del Taller 1 (tiempo, producto, región), para evitar colisión con las tablas `dim_tiempo`, `dim_producto` y `dim_region` creadas previamente en la misma base `ejerciciodw` — un problema real que se documenta en la Sección 5.

4 Flujo ETL en KNIME

El flujo completo se construyó en el workflow `Taller1_Parte5` de KNIME Analytics Platform, organizado en tres etapas: extracción, transformación y carga.

4.1 Extracción (nodos de conexión y lectura)

Fuente	Nodos utilizados
CSV (calendario)	CSV Reader
MySQL	MySQL Connector → 3× DB Query Reader (ventas+puente, productos, regiones)
PostgreSQL	PostgreSQL Connector → DB Query Reader (clientes)
MongoDB	MongoDB Connector → MongoDB Reader → JSON to Table (canal_venta)

4.2 Transformación (nodos de KNIME)

Nodo	Transformación aplicada
String to Date&Time	Convierte la columna <code>fecha</code> del CSV (texto) a tipo Fecha real, necesario para poder unirla con <code>fecha_venta</code> de MySQL.
Joiner	Une la base de ventas+puente (MySQL) con el calendario (CSV) por fecha, para obtener <code>id_tiempo</code> .
Column Filter (×3)	Reduce cada tabla intermedia a solo las columnas necesarias para la dimensión o el hecho final (por ejemplo, descarta <code>\$oid</code> de MongoDB, o las columnas de fecha duplicadas tras el Joiner).
String Manipulation	Limpieza de texto: <code>strip(\$categoria_producto\$)</code> elimina espacios sobrantes en la categoría de producto.

4.3 Carga

Se usaron 6 nodos `DB Writer`, cada uno con dos entradas: los datos ya transformados, y la conexión de destino (`MySQL Connector` hacia `ejerciciodw`) — incluso para las dimensiones cuyo origen fue PostgreSQL o MongoDB, ya que el Data Warehouse final se centralizó en MySQL. Todas las cargas se hicieron en modo `Append rows`.

Home

Taller1_Parte5

+

?

Preferences

Menu

Sign in

Save

↶

↷

Execute

Cancel

Reset

Local - Taller1_Parte5

Upload

Info

Nodes

DB Writer

DB

Read/Write

Write

Query

IO

+22

DB Writer

MongoDB Writer

DB Writer (DB Data)

CSV Writer

XML Writer

JSON Writer

PMML Writer

ARFF Writer

Excel Writer

Table Writer

Model Writer

Explorer Writer

Report PDF Writer

DB Sorter

Report HTML Writer

DB Updater

Image Writer (Part)

Google Sheets Writer

Image Writer (Table)

Distance Matrix Writer

DB Query

DB Merger

DB Row Filter

DB Row Deleter

DB Row Inserter

DB Reader

DB Joiner

DB Loader

DB Looper

DB Binner

DB Table Deleter

JSON Row Combiner and...

DB Column Filter

DB Writer

Database Table

Schema name

Table name

dim_canal_venta_p5

Browse

If table exists in database

Append rows

Remove existing table and contained data

Fail

Changing

Columns to write (SET in SQL)

Manual

Pattern

Type

Search

Excludes

Includes

id_canal

nombre_canal

tipo

Discard

Apply

Apply and Execute

1: Input Data with Write Status

2: DB Data

Flow Variables

Table

Statistics

Search

HH

#	RowID	id_canal	nombre_canal	tipo	Write Status	Write Error
		Number (Integer)	String	String	Number (Integer)	String
1	Row0	1	Tienda Física	Presencial	1	
2	Row1	2	En Línea	Digital	1	
3	Row2	3	Teléfono	Presencial	1	
4	Row3	4	Marketplace	Digital	1	

Figura 1. Vista general del workflow Taller1_Parte5 en KNIME: 4 fuentes, nodos de transformación y 6 nodos de carga hacia MySQL.

5 Problemas reales encontrados y solución

Durante la construcción del flujo se presentaron varios problemas concretos, documentados aquí porque su resolución fue parte relevante del proceso:

BUG 1 — CHOQUE DE TIPOS EN EL JOINER

El `Joiner` entre el CSV de calendario y la tabla de ventas de MySQL inicialmente devolvía **0 filas**. La causa: `fecha` (CSV) llegaba como texto (String) mientras que `fecha_venta` (MySQL) era de tipo Fecha real. KNIME no las consideraba iguales aunque se vieran idénticas. **Solución:** insertar un nodo `String to Date&Time` antes del Joiner para convertir explícitamente la columna del CSV al tipo correcto.

BUG 2 — FUNCIÓN TRIM() NO DISPONIBLE

El nodo `String Manipulation` rechazó la función `trim(str)` con un error de compilación (método no encontrado), a pesar de ser una función estándar en la documentación de KNIME. **Solución:** se localizó el equivalente exacto en la lista de funciones del propio editor — `strip(str)` — y se usó ese en su lugar.

BUG 3 — COLISIÓN DE NOMBRES CON EL TALLER 1

Al cargar la dimensión de tiempo con el nombre `dim_tiempo`, MySQL devolvió `Duplicate entry '32' for key 'dim_tiempo.PRIMARY'`: esa tabla ya existía, creada en el Taller 1 (Parte 4), y compartía el mismo rango de IDs. Un segundo intento de sobrescribirla también falló porque estaba referenciada por una llave foránea desde `hechos_ventas` (`Cannot drop table 'dim_tiempo' referenced by a foreign key constraint`). **Solución:** renombrar las dimensiones nuevas que replican conceptos del Taller 1 con el sufijo `_p5` (`dim_tiempo_p5`, `dim_producto_p5`, `dim_region_p5`), manteniendo ambos talleres independientes dentro de la misma base `ejerciciodw`.

BUG 4 — SESIÓN DE CONEXIÓN PERDIDA

Los nodos `DB Writer` reportaban `Database not connected` aun con la línea de conexión dibujada desde el `MySQL Connector`. **Solución:** re-ejecutar el nodo `MySQL Connector` para refrescar la sesión antes de ejecutar los nodos que dependen de él.

6 Verificación de resultados

Cada tabla de destino se verificó tanto desde el propio panel de salida de KNIME (columna `Write Status = 1` en todas las filas, sin errores) como de forma independiente desde la extensión Database Client de VSCode, confirmando el conteo final de 17 tablas en la base `ejerciciodw` (11 del Taller 1 + 6 nuevas de la Parte 5).

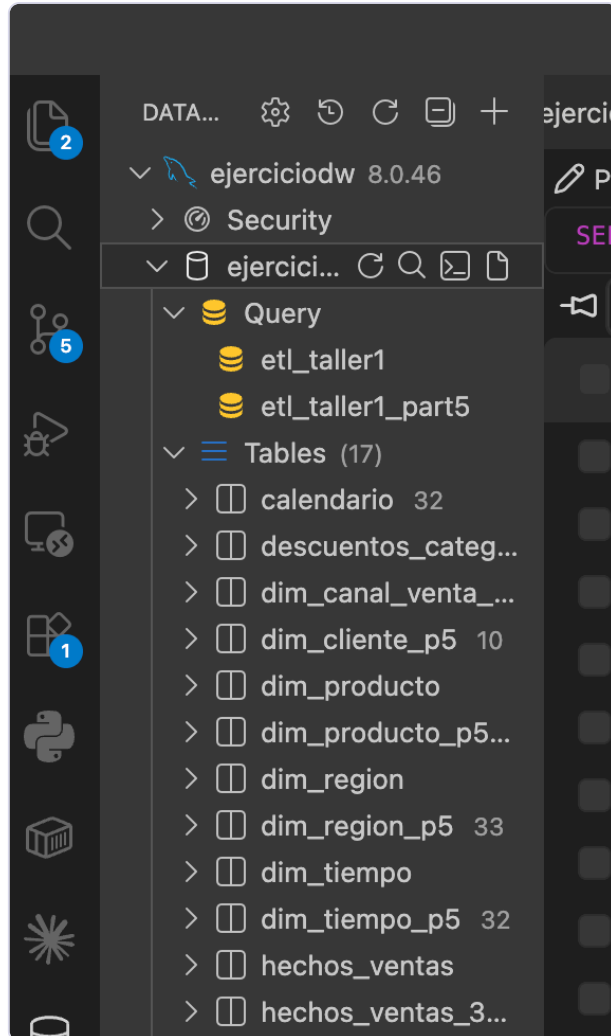


Figura 2. Verificación final desde VSCode (Database Client): las 6 tablas de la Parte 5 (*dim_canal_venta_p5*, *dim_cliente_p5*, *dim_producto_p5*, *dim_region_p5*, *dim_tiempo_p5*, *hechos_ventas_360*) junto a las 11 tablas del Taller 1 — 17 tablas en total.

7 Conclusiones

El ejercicio permitió integrar en un solo flujo cuatro tecnologías de almacenamiento distintas (archivo plano, dos motores relacionales y una base documental), replicando un escenario realista donde los datos de una organización rara vez viven en un único sistema. Los problemas encontrados — de tipos de datos, de sintaxis específica de KNIME, y de colisión de nombres entre entregas del curso — reflejan dificultades típicas de un proceso ETL real, y su documentación aquí sirve como referencia para resolver casos similares en el futuro.